Project Writeup

Matthew Muth | 971-727-0402

For this project, I chose to use Python to build a classification model capable of taking the provided training/test data and outputting sensible make probabilities. In previous experience, I had not worked with compressed data. On that note, my first steps were to do some data exploration and cleaning within the dataset. This process included checking for any null values within the columns and checking data types. During this search I honed in on ‘closestdefapproach’ and ‘distcont1...4’ as columns that might cause some hiccups when training the model. This lead me to the decision to break up the array of time-to-contest within ‘closestdefapproch’. After cleaning up defender data, the last part of my preprocess was to calibrate court location. This was my first mistake that I will explain later, but I added court dimensions and shot zones after running a few training rounds.

shooterspeed --- f64	closestdefdist --- f64	distcont1 --- str	distcont2 --- str	distcont3 --- str	distcont4 --- str
2.386667	11.332678	NA	NA	NA	NA
4.470777	2.969714	2.9697137909	NA	NA	NA
8.414604	2.556345	2.5563450471	2.6940861159	NA	NA
12.971724	3.689255	NA	NA	NA	NA
0.041667	9.19981	9.1998097806	NA	NA	NA
…	…	…	…	…	…
0.515388	5.253808	5.2538081427	NA	NA	NA
13.582055	3.612035	4.4058143402	NA	NA	NA
3.29483	3.749733	3.7497333239	5.0155159256	NA	NA
0.971825	6.198847	6.1988466669	NA	NA	NA
0.658808	9.048342	9.0483423896	NA	NA	NA

Building the Model

After loading in the data and creating my preprocess functions it was time to build out the model. Because we want our model to make predictions on shot make probability, our target will be the ‘outcome’ column. Going off of that, we want our model trained on features that affect shot making. I chose to drop id columns used to identify the shooter, defender, or the closest contest. The possible side effect of this decision is that my model won’t take individual player data into account but I believe a broader approach made a better model with the data I was given. Next was converting categorical columns to dummy variables as part of the encoding process.

Training the Model

To start training the model I needed to know the best parameters for this specific dataset. By importing xgboost and sklearn python packages, I initiated a randomized search for n estimators, max tree depth, and learning rate. The evaluation metric was log-loss which is what the final submission will be graded against. This initial search recommended a max depth of 8 combined with a learning rate of 1.0 which caused some concern with overfitting the model.

```
param_grid = {
    'n_estimators': [200, 400, 600],
    'max_depth': [3, 4, 5, 6, 8],
    'learning_rate': [0.01, 0.03, 0.05, 0.1],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
}

search = RandomizedSearchCV(
    XGBClassifier(eval_metric='logloss'),
    param_distributions=param_grid,
    n_iter=30,
    scoring='neg_log_loss',
    cv=5,
    random_state=42,
    n_jobs=-1
)
search.fit(X_train, y_train)

print(search.best_params_)
best_model = search.best_estimator_
```

Using these parameters, I did my first round of training to see where the model would land and if overfitting would be an issue. Results showed expected and realistic make probabilities but raised some alarming log-loss, Briar, and AUC scores that pointed me towrds going back in and revaluating the paramters. At this point was when I dove deeper into the location features and built out shot zone functions that implemented the coordinates given in the reference image. To narrow down better parameters for the final train, I used a train/test split on the visible data within the test file. This approach gave a much lower log-loss number than the first test due to overfitting. Now that I knew the depth of 8 was too much, I needed to find the best possible parameters for the final run. Further testing gave me final parameters I was confident in. Before committing to the final training run, I wanted to double check how much each feature was affecting the model. Using pandas to look at the feature importances, I was able to see which features influenced the model the most. Finally, with inside info on what the model is looking at and final changes to training parameters I was able to export final probabilities into the submission csv.

```
Log loss: 0.6161277294158936
Brier score: 0.21543869376182556
AUC: 0.6922191508948914
shot_zone_rim              0.230241
shottype_dunk              0.210898
contested                  0.073619
shottype_jumper            0.060460
three                      0.048880
distance                   0.030362
shottype_layup             0.025348
shot_zone_key_three        0.024908
num_contesters             0.024620
shot_zone_corner_three     0.023110
distcont1                  0.016636
shottype_tip               0.014708
shot_zone_short_mid        0.012693
shot_zone_mid_range        0.011070
distcont2                  0.008486
dtype: float32
```

Reflection

With more time to work on this project I would obviously want to go deeper into the data to make the best model possible. But at the same time, I am confident in where the model finished. Some things I might do differently would be looking closer into specific player data to see if the model can pick up on any patterns that would increase or decrease shot probabilities. As I mentioned earlier, I made some mistakes early in the project. In future projects, I will get all preprocess functions and data cleaning done before starting any model training. I am grateful for the experience this project gave me and a look at what is being built in the industry.